Lab 06. Region based Segmentation

Computer Vision 2025-2026

Mục lục

1 Mục tiêu của bài thực hành **3**

2 Kiến thức nền đã có và cách kết nối vào phân đoạn **3**

2.1 Ảnh đa cấp xám và histogram 3

2.2 Các thao tác trên điểm ảnh 3

2.3 Phép toán số học, logic và morphology 4

2.4 Filtering và edge detection 4

3 Sơ đồ pipeline tổng quát **4**

4 Các hàm dùng chung cho toàn bài **5**

4.1 Thư viện sử dụng 5

4.2 Đọc ảnh, chuyển sang ảnh xám và mảng số 5

4.3 Hiển thị một hoặc nhiều ảnh 6

4.4 Hiển thị mask, distance map, markers, cluster labels 6

4.5 Chồng kết quả lên ảnh gốc 7

4.6 Hiển thị biên watershed lên ảnh 8

4.7 Hiển thị cụm K-means và quadtree của Split–Merge 8

5 Phương pháp 1: K-means segmentation **9**

5.1 Bản chất của phương pháp 9

5.2 Hàm mục tiêu 9

5.3 Khi nào K-means hoạt động tốt? 9

5.4 Pipeline gợi ý cho K-means 10

5.5 Tự cài đặt K-means bằng NumPy: chỉ dùng cường độ sáng 10

5.6 Mở rộng: K-means dùng cường độ và tọa độ 11

5.7 Dùng OpenCV cho K-means 12

5.8 Ví dụ minh họa trực quan 12

5.9 Gợi ý loại ảnh nên dùng để minh họa 12

5.10 Bài tập mở rộng cho K-means 12

6 Phương pháp 2: Region Growing **13**
6.1 Ý tưởng và bản chất . . . 13
6.2 Các thành phần phải xác định . . . 14
6.3 Pipeline gợi ý cho Region Growing . . . 14
6.4 Tự cài đặt Region Growing: so với intensity của seed . . . 14
6.5 Tự cài đặt Region Growing: so với trung bình vùng hiện tại . . . 15
6.6 Seed tốt và seed xấu . . . 16
6.7 Gợi ý loại ảnh nên dùng để minh họa . . . 16
6.8 Bài tập mở rộng cho Region Growing . . . 16

7 Phương pháp 3: Watershed và Distance Transform **17**
7.1 Ý tưởng trực quan của watershed . . . 17
7.2 Distance Transform là gì và tại sao quan trọng? . . . 18
7.3 Pipeline đầy đủ cho Watershed dùng marker từ distance transform . . . 18
7.4 Code đầy đủ cho pipeline Watershed với OpenCV . . . 18
7.5 Hiển thị toàn bộ các kết quả trung gian . . . 19
7.6 Marker tốt và marker xấu . . . 20
7.7 Gợi ý loại ảnh nên dùng để minh họa . . . 20
7.8 Bài tập mở rộng cho Watershed . . . 20

8 Phương pháp 4: Split–Merge segmentation **21**
8.1 Ý tưởng . . . 21
8.2 Tiêu chí đồng nhất . . . 22
8.3 Pipeline gợi ý cho Split–Merge . . . 22
8.4 Code đầy đủ phần split theo quadtree . . . 22
8.5 Gán nhãn các ô split để trực quan . . . 23
8.6 Merge các ô kề nhau theo trung bình mức xám . . . 23
8.7 Ví dụ minh họa Split–Merge . . . 24
8.8 Gợi ý loại ảnh nên dùng để minh họa . . . 24
8.9 Bài tập mở rộng cho Split–Merge . . . 25

9 So sánh tổng hợp giữa các phương pháp **25**

10 Các bài tập tổng hợp cho toàn bộ bài học **26**

11 Hướng dẫn viết báo cáo sau buổi lab **28**

12 Kết luận của bài học **29**

1. Mục tiêu của bài thực hành

Sau bài thực hành này, sinh viên cần đạt được các mục tiêu sau:

- Ôn lại và kết nối các kiến thức đã học: ảnh đa cấp xám, histogram, thresholding, phép toán điểm ảnh, phép toán số học/logic, bộ lọc, phát hiện biên, morphology.
- Hiểu bản chất của các phương pháp phân đoạn dựa trên vùng: **K-means**, **Region Growing**, **Watershed**, **Split–Merge**.
- Phân biệt rõ đầu vào, đầu ra, giả thiết hoạt động, ưu nhược điểm của từng thuật toán.
- Thấy được vai trò quan trọng của **seed** và **marker**; hiểu vì sao chọn tốt hay xấu lại dẫn tới kết quả khác nhau.
- Hiểu ý nghĩa của **distance transform** trong việc tách các đối tượng chạm nhau.
- Tự xây dựng pipeline xử lý hoàn chỉnh bằng thư viện cơ sở như `NumPy` để nắm bản chất từng bước, đồng thời biết dùng các thư viện có sẵn như `OpenCV` để triển khai nhanh hơn.
- Biết cách thiết kế thí nghiệm, chọn ảnh đầu vào phù hợp và so sánh kết quả giữa các phương pháp.

2. Kiến thức nền đã có và cách kết nối vào phân đoạn

Trong các bài trước, các bạn đã học nhiều thành phần quan trọng của xử lý ảnh. Phần phân đoạn ảnh dựa trên vùng không tách rời những thành phần đó, mà sử dụng chúng như các khối xây dựng trong một pipeline đầy đủ.

2.1. Ảnh đa cấp xám và histogram

Đối với ảnh xám, mỗi pixel có một mức cường độ sáng. Histogram cho biết phân bố các mức sáng trong ảnh. Khi histogram có các cụm tách biệt tương đối rõ, các kỹ thuật threshold hoặc clustering thường hoạt động tốt. Nếu histogram bị chồng lấn mạnh, chỉ dùng một ngưỡng toàn cục thường không đủ, khi đó cần dùng thêm thông tin lân cận, thông tin vùng hoặc gradient.

2.2. Các thao tác trên điểm ảnh

Các phép tăng giảm sáng, hiệu chỉnh gamma, chuẩn hóa ảnh, thresholding và auto-thresholding có vai trò rất lớn trước khi phân đoạn. Ví dụ, nếu ảnh quá tối hoặc quá sáng, K-means trên cường độ hay Region Growing theo ngưỡng độ giống nhau đều sẽ kém ổn định. Một bước chuẩn hóa đơn giản trước đó có thể cải thiện rõ kết quả.

2.3. Phép toán số học, logic và morphology

Sau khi tạo được một mask nhị phân thô, ta thường cần:

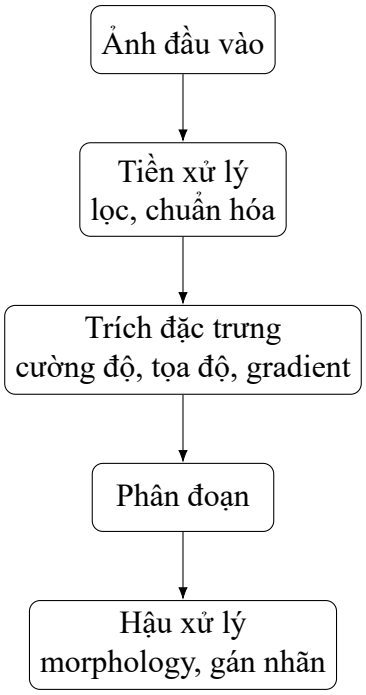
- phép toán logic để kết hợp nhiều mask,
- opening hoặc closing để loại nhiễu nhỏ,
- erosion, dilation để co giãn vùng,
- connected components để tách từng đối tượng.

2.4. Filtering và edge detection

Nhiễu ảnh hưởng mạnh tới mọi thuật toán phân đoạn. Gaussian filter thường dùng để làm mượt nhiễu Gaussian; median filter hữu ích với nhiễu muối tiêu. Với watershed, gradient hoặc biên thường được dùng như một bề mặt địa hình; do đó bộ lọc và edge detection có thể được dùng để tạo đầu vào tốt hơn cho bước phân đoạn.

3. Sơ đồ pipeline tổng quát

Một pipeline điển hình của phân đoạn ảnh cổ điển có thể được tóm tắt như sau:



Điểm quan trọng là: **không có thuật toán nào luôn tốt nếu đầu vào chưa được chuẩn bị phù hợp**. Vì vậy, trong bài lab này, mỗi phương pháp đều được trình bày theo ba lớp:

1. **Giải thích bản chất và điều kiện hoạt động tốt**,
2. **Pipeline gợi ý trong ứng dụng**,
3. **Code minh họa đầy đủ** bằng cả cách tự cài đặt và dùng thư viện có sẵn.

4. Các hàm dùng chung cho toàn bài

Phần này trình bày các hàm dùng chung để đọc ảnh, chuyển sang mảng xám và hiển thị các kết quả trực quan. Ý tưởng là các bạn có thể dùng chung các hàm này cho mọi thuật toán, thay vì mỗi phần lại tự viết lại từ đầu.

4.1. Thư viện sử dụng

Trong toàn bộ bài lab, ta giả sử sử dụng các thư viện sau:

```
import os
import cv2
import math
import numpy as np
import matplotlib.pyplot as plt
from collections import deque
from matplotlib import patches
```

Listing 1: Các thư viện cơ bản dùng trong toàn bài lab

4.2. Đọc ảnh, chuyển sang ảnh xám và mảng số

```
def read_gray_image(path, to_float=False):
    """Đọ
    c ảnh ừt đĩa và ảtr ềv ảnh xám ạdng NumPy array.

    Input:
        path: đường ẫdn ảnh
        to_float: ếnu True thì ẩchun hóa ềv [0, 1]
    Output:
        gray: ảmng 2D ảnh xám
    """
    img = cv2.imread(path, cv2.IMREAD_GRAYSCALE)
    if img is None:
        raise FileNotFoundError(f"Không đọc được ảnh: {path}")
    if to_float:
        return img.astype(np.float32) / 255.0
    return img


def bgr_to_gray_array(img_bgr, to_float=False):
    """ểChuyn ảnh BGR (ếnu đã đọc ẫsn) sang ảnh xám."""
    gray = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2GRAY)
    if to_float:
        return gray.astype(np.float32) / 255.0
    return gray
```

Listing 2: Các hàm đọc ảnh và chuẩn hóa về ảnh xám

4.3. Hiển thị một hoặc nhiều ảnh

```
def show_gray(img, title='Gray image', figsize=(5, 5), cmap='gray'):
    plt.figure(figsize=figsize)
    plt.imshow(img, cmap=cmap)
    plt.title(title)
    plt.axis('off')
    plt.tight_layout()
    plt.show()


def show_images(images, titles=None, cols=3, figsize=(14, 8), cmap='
    gray'):
    n = len(images)
    rows = math.ceil(n / cols)
    plt.figure(figsize=figsize)

    for i, img in enumerate(images):
        plt.subplot(rows, cols, i + 1)
        if img.ndim == 2:
            plt.imshow(img, cmap=cmap)
        else:
            plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB))
        plt.axis('off')
        if titles is not None and i < len(titles):
            plt.title(titles[i])

    plt.tight_layout()
    plt.show()
```

Listing 3: Các hàm hiển thị ảnh cơ bản bằng Matplotlib

4.4. Hiển thị mask, distance map, markers, cluster labels

Vì các kiểu kết quả trung gian trong segmentation khác nhau, ta cần các hàm hiển thị chuyên biệt.

```
def show_binary_mask(mask, title='Binary mask'):
    plt.figure(figsize=(5, 5))
    plt.imshow(mask, cmap='gray')
    plt.title(title)
    plt.axis('off')
    plt.tight_layout()
    plt.show()


def show_distance_map(dist_map, title='Distance transform', cmap='
    viridis'):
    plt.figure(figsize=(5, 5))
    plt.imshow(dist_map, cmap=cmap)
    plt.colorbar(fraction=0.046, pad=0.04)
    plt.title(title)
    plt.axis('off')
```

```
    plt.tight_layout()
    plt.show()


def colorize_label_map(label_map):
    """ể
    Chuyn label map (0,1,2,...) sang ảnh màu để quan sát ựtrc quan.
    """
    label_map = label_map.astype(np.int32)
    max_label = label_map.max()
    if max_label <= 0:
        return np.zeros((*label_map.shape, 3), dtype=np.uint8)

    rng = np.random.default_rng(0)
    colors = rng.integers(0, 255, size=(max_label + 1, 3), dtype=np.
       uint8)
    colors[0] = np.array([0, 0, 0], dtype=np.uint8)
    return colors[label_map]


def show_label_map(label_map, title='Label map'):
    color = colorize_label_map(label_map)
    plt.figure(figsize=(5, 5))
    plt.imshow(color)
    plt.title(title)
    plt.axis('off')
    plt.tight_layout()
    plt.show()


def show_markers(markers, title='Markers'):
    plt.figure(figsize=(5, 5))
    plt.imshow(markers, cmap='nipy_spectral')
    plt.colorbar(fraction=0.046, pad=0.04)
    plt.title(title)
    plt.axis('off')
    plt.tight_layout()
    plt.show()
```

Listing 4: Hiển thị mask nhị phân, distance map, markers, labels

4.5. Chồng kết quả lên ảnh gốc

Phần code này giúp sinh viên thấy kết quả nằm ở đâu trên ảnh ban đầu.

```
def overlay_mask_on_gray(gray, mask, color=(255, 0, 0), alpha=0.35):
    """ồ
    Chng mask ịnh phân lên ảnh xám.
    gray: ảnh xám uint8
    mask: ảnh ịnh phân 0/1 ặhoc 0/255
    """
    if gray.dtype != np.uint8:
```

```
        gray_u8 = np.clip(gray * 255 if gray.max() <= 1.0 else gray, 0,
            255).astype(np.uint8)
    else:
        gray_u8 = gray.copy()

    rgb = cv2.cvtColor(gray_u8, cv2.COLOR_GRAY2RGB).astype(np.float32)
    mask_bool = mask > 0

    overlay = rgb.copy()
    overlay[mask_bool] = (1 - alpha) * overlay[mask_bool] + alpha * np.
       array(color, dtype=np.float32)
    return np.clip(overlay, 0, 255).astype(np.uint8)
```

Listing 5: Phủ màu kết quả phân đoạn lên ảnh gốc

4.6. Hiển thị biên watershed lên ảnh

```
def draw_watershed_boundaries(gray, markers, boundary_value=-1):
    if gray.dtype != np.uint8:
        gray_u8 = np.clip(gray * 255 if gray.max() <= 1.0 else gray, 0,
            255).astype(np.uint8)
    else:
        gray_u8 = gray.copy()

    rgb = cv2.cvtColor(gray_u8, cv2.COLOR_GRAY2RGB)
    rgb[markers == boundary_value] = [255, 0, 0]
    return rgb
```

Listing 6: Tô biên watershed lên ảnh

4.7. Hiển thị cụm K-means và quadtree của Split–Merge

```
def show_kmeans_clusters(label_map, title='K-means clusters'):
    show_label_map(label_map, title=title)


def draw_quadtree_boxes(gray, rectangles, title='Quadtree split result'
   ):
    plt.figure(figsize=(6, 6))
    plt.imshow(gray, cmap='gray')
    ax = plt.gca()

    for (x, y, w, h) in rectangles:
        rect = patches.Rectangle((y, x), w, h, linewidth=1.2,
                                 edgecolor='red', facecolor='none')
        ax.add_patch(rect)

    plt.title(title)
    plt.axis('off')
    plt.tight_layout()
```

```
    plt.show()
```

Listing 7: Hiển thị label K-means và các ô split của quadtree

5. Phương pháp 1: K-means segmentation

5.1. Bản chất của phương pháp

K-means là một thuật toán phân cụm không giám sát. Trong segmentation, ta xem mỗi pixel là một mẫu dữ liệu, rồi chia tất cả các mẫu đó thành K cụm sao cho các pixel trong cùng một cụm "giống nhau" hơn so với các pixel ở cụm khác.

Điểm quan trọng cần làm rõ ngay từ đầu là: **mẫu đầu vào của K-means là gì?**

Có hai cách thường gặp:

1. **Chỉ dùng cường độ sáng**: mỗi pixel được biểu diễn bởi một giá trị $[I]$.
2. **Dùng cường độ sáng kết hợp tọa độ**: mỗi pixel được biểu diễn bởi $[I, x, y]$ hoặc $[I, \lambda x, \lambda y]$.

Nếu chỉ dùng cường độ sáng, K-means chỉ nhóm các pixel có độ sáng gần nhau, bất kể chúng nằm ở đâu. Điều này phù hợp khi ảnh có histogram tách biệt rõ, nhưng có thể làm hai vùng ở xa nhau mà cùng sáng bị gộp chung. Nếu bổ sung tọa độ, thuật toán bắt đầu "quan tâm" đến cấu trúc không gian, từ đó tạo ra các vùng cụm gọn hơn về mặt hình học.

5.2. Hàm mục tiêu

Với N mẫu đầu vào $\mathbf{x}_1, \ldots, \mathbf{x}_N$, K-means tìm K tâm cụm $\boldsymbol{\mu}_1, \ldots, \boldsymbol{\mu}_K$ sao cho tổng sai số bình phương trong cụm là nhỏ nhất:

$$\min \sum_{k=1}^{K} \sum_{\mathbf{x}_i \in C_k} \|\mathbf{x}_i - \boldsymbol{\mu}_k\|^2.$$

Thuật toán lặp qua hai bước cơ bản:

- **Assignment step**: gán mỗi pixel vào tâm cụm gần nhất.
- **Update step**: tính lại tâm cụm bằng trung bình các pixel hiện thuộc cụm đó.

5.3. Khi nào K-means hoạt động tốt?

K-means phù hợp khi:

- các vùng có mức sáng khác nhau tương đối rõ,
- ảnh ít nhiễu,
- ta cần một phân hoạch nhanh, đơn giản, dễ quan sát.

K-means kém hiệu quả khi:

- các vùng có cùng mức sáng nhưng khác vị trí,
- biên vùng phức tạp,
- đối tượng chồng nhau hoặc dính nhau,
- số cụm K khó chọn.

5.4. Pipeline gợi ý cho K-means

Pipeline cơ bản cho K-means

Ảnh xám $\rightarrow$ lọc nhiễu nhẹ $\rightarrow$ tạo vector đặc trưng I hoặc (I, x, y) $\rightarrow$ chạy K-means $\rightarrow$ thu được label map $\rightarrow$ chọn cụm quan tâm hoặc hiển thị toàn bộ cụm.

5.5. Tự cài đặt K-means bằng NumPy: chỉ dùng cường độ sáng

```python
def kmeans_gray_intensity(gray, k=3, max_iter=30, tol=1e-4, seed=0):
    """
    K-means trên ảnh xám ớvi đặc ưtrng ỗmi pixel là [I].

    Input:
        gray: ảnh xám 2D
        k: ốs ụcm
    Output:
        label_map: nhãn ụcm cho ừtng pixel
        centers: tâm ụcm theo intensity
    """
    X = gray.astype(np.float32).reshape(-1, 1)
    rng = np.random.default_rng(seed)

    init_idx = rng.choice(X.shape[0], size=k, replace=False)
    centers = X[init_idx].copy()

    for _ in range(max_iter):
        dists = np.sum((X[:, None, :] - centers[None, :, :]) ** 2, axis
            =2)
        labels = np.argmin(dists, axis=1)

        new_centers = centers.copy()
        for c in range(k):
            pts = X[labels == c]
            if len(pts) > 0:
                new_centers[c] = pts.mean(axis=0)

        shift = np.linalg.norm(new_centers - centers)
        centers = new_centers
        if shift < tol:
```

```
            break

    label_map = labels.reshape(gray.shape)
    return label_map, centers.reshape(-1)
```

Listing 8: K-means chỉ dùng cường độ sáng làm đặc trưng

5.6. Mở rộng: K-means dùng cường độ và tọa độ

Khi thêm tọa độ (x, y), cần chú ý rằng thang đo của cường độ và tọa độ là khác nhau. Thông thường ta chuẩn hóa cường độ về $[0, 1]$ và chuẩn hóa tọa độ theo kích thước ảnh. Ta cũng có thể nhân tọa độ với một hệ số λ để điều chỉnh tầm quan trọng của yếu tố không gian.

```
def kmeans_gray_intensity_xy(gray, k=3, spatial_weight=0.25,
                             max_iter=30, tol=1e-4, seed=0):
    h, w = gray.shape
    yy, xx = np.mgrid[0:h, 0:w]

    I = gray.astype(np.float32) / 255.0
    X = np.stack([
        I,
        spatial_weight * (yy.astype(np.float32) / max(h - 1, 1)),
        spatial_weight * (xx.astype(np.float32) / max(w - 1, 1))
    ], axis=-1).reshape(-1, 3)

    rng = np.random.default_rng(seed)
    init_idx = rng.choice(X.shape[0], size=k, replace=False)
    centers = X[init_idx].copy()

    for _ in range(max_iter):
        dists = np.sum((X[:, None, :] - centers[None, :, :]) ** 2, axis
           =2)
        labels = np.argmin(dists, axis=1)

        new_centers = centers.copy()
        for c in range(k):
            pts = X[labels == c]
            if len(pts) > 0:
                new_centers[c] = pts.mean(axis=0)

        shift = np.linalg.norm(new_centers - centers)
        centers = new_centers
        if shift < tol:
            break

    label_map = labels.reshape(gray.shape)
    return label_map, centers
```

Listing 9:]K-means trên đặc trưng [I, lambda*x, lambda*y]

5.7. Dùng OpenCV cho K-means

```
def kmeans_opencv_intensity(gray, k=3, attempts=10):
    X = gray.reshape(-1, 1).astype(np.float32)
    criteria = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
                50, 0.2)
    compactness, labels, centers = cv2.kmeans(
        X, k, None, criteria, attempts, cv2.KMEANS_PP_CENTERS
    )
    label_map = labels.reshape(gray.shape)
    return label_map, centers.reshape(-1), compactness
```

Listing 10: Dùng OpenCV để chạy K-means trên cường độ sáng

5.8. Ví dụ minh họa trực quan

```
gray = read_gray_image('sample_images/coins.png')
label_map_1, centers_1 = kmeans_gray_intensity(gray, k=3)
label_map_2, centers_2 = kmeans_gray_intensity_xy(gray, k=3,
   spatial_weight=0.35)

show_images(
    [gray, colorize_label_map(label_map_1), colorize_label_map(
       label_map_2)],
    ['Ảnh ốgc', 'K-means ỉch intensity', 'K-means intensity + ọta độ'],
    cols=3, figsize=(15, 5), cmap='gray'
)
```

Listing 11: Ví dụ chạy và hiển thị cụm K-means

5.9. Gợi ý loại ảnh nên dùng để minh họa

Để làm rõ sự khác nhau giữa hai cách chọn đặc trưng đầu vào, nên chọn các loại ảnh sau:

- Ảnh có các vùng sáng tối khá rõ, ví dụ ảnh đồng xu trên nền tối, ảnh vật thể đơn giản trên nền phẳng.
- Ảnh có nhiều vùng ở vị trí khác nhau nhưng cùng mức xám, để thấy K-means chỉ theo intensity dễ gộp sai.
- Ảnh có gradient nền chậm, để xem việc thêm tọa độ có làm cụm ổn định hơn hay không.

5.10. Bài tập mở rộng cho K-means

Bài K1: So sánh đặc trưng đầu vào

Yêu cầu: Chạy K-means với hai kiểu đặc trưng: $[I]$ và $[I, x, y]$.

Đầu vào: Một ảnh xám có ít nhất hai vùng có mức sáng gần giống nhau nhưng ở vị trí khác nhau.

Đầu ra: Hai label map và ảnh hiển thị cụm màu.

Mục đích: Thấy rõ khi nào việc thêm tọa độ giúp phân đoạn ổn định hơn.

So sánh cần làm: So sánh trực quan cụm nhận được; mô tả vùng nào bị gộp nhầm khi không dùng tọa độ.

Bài K2: Ảnh hưởng của số cụm K

Yêu cầu: Thử $K = 2, 3, 4, 5$ trên cùng một ảnh.

Đầu vào: Ảnh có nền và nhiều đối tượng với số mức sáng khác nhau.

Đầu ra: Bộ ảnh cluster map tương ứng với từng giá trị K.

Mục đích: Hiểu rằng chọn K quá nhỏ sẽ gộp vùng khác nhau; chọn quá lớn sẽ phân mảnh quá mức.

So sánh cần làm: Nhận xét cụm nào có ý nghĩa vật lý, cụm nào chỉ là phân mảnh nhân tạo.

Bài K3: Kết hợp tiền xử lý

Yêu cầu: So sánh K-means trước và sau khi lọc Gaussian hoặc median.

Đầu vào: Ảnh có nhiễu nhẹ hoặc biên không đều.

Đầu ra: Kết quả clustering trước lọc và sau lọc.

Mục đích: Thấy vai trò của filtering đối với phân cụm.

So sánh cần làm: Đánh giá vùng nhiễu có bị phân thành cụm riêng hay không.

Bài K4: Tạo mask đối tượng từ cluster map

Yêu cầu: Sau khi phân cụm, chọn một hoặc nhiều cụm để tạo mask nhị phân cho đối tượng quan tâm.

Đầu vào: Ảnh trong đó đối tượng quan tâm có mức sáng khác nền.

Đầu ra: Mask nhị phân, overlay trên ảnh gốc.

Mục đích: Thấy cách K-means được dùng như một bước trong pipeline segmentation hoàn chỉnh chứ không chỉ để tô màu các cụm.

6. Phương pháp 2: Region Growing

6.1. Ý tưởng và bản chất

Region Growing bắt đầu từ một hoặc nhiều điểm hạt giống (seed), sau đó mở rộng vùng bằng cách thêm các pixel lân cận nếu chúng "đủ giống" với vùng hiện tại theo một tiêu chí nào đó.

Điểm mạnh của Region Growing là rất trực quan: nếu ta chọn đúng seed ở trong đối tượng và tiêu chí giống nhau hợp lý, vùng sẽ "lan" ra theo đúng đối tượng. Tuy nhiên, nếu seed đặt sai hoặc ảnh nhiễu, vùng có thể lan sang nền hoặc lan thiếu.

6.2. Các thành phần phải xác định

Để dùng Region Growing, cần trả lời rõ bốn câu hỏi:

1. Seed lấy ở đâu?
2. Dùng lân cận 4 hướng hay 8 hướng?
3. Tiêu chí giống nhau là so với **seed**, so với **pixel hiện tại**, hay so với **trung bình vùng**?
4. Khi nào dừng?

Trong lab này, ta sẽ minh họa cả hai biến thể thông dụng:

- so với cường độ tại seed ban đầu,
- so với trung bình vùng đang phát triển.

6.3. Pipeline gợi ý cho Region Growing

Pipeline cơ bản cho Region Growing

Ảnh xám → lọc nhiễu nhẹ → chọn seed thủ công hoặc tự động → mở rộng vùng theo tiêu chí đồng nhất → thu được mask → hậu xử lý bằng morphology nếu cần.

6.4. Tự cài đặt Region Growing: so với intensity của seed

```python
def region_growing_seed_ref(gray, seed, threshold=8, connectivity=4):
    h, w = gray.shape
    visited = np.zeros((h, w), dtype=bool)
    region = np.zeros((h, w), dtype=np.uint8)

    seed_x, seed_y = seed
    seed_val = int(gray[seed_x, seed_y])
    q = deque([(seed_x, seed_y)])

    if connectivity == 8:
        neighbors = [(-1, -1), (-1, 0), (-1, 1),
                     (0, -1),           (0, 1),
                     (1, -1),  (1, 0),  (1, 1)]
    else:
        neighbors = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while q:
        x, y = q.popleft()
        if visited[x, y]:
            continue
        visited[x, y] = True

        if abs(int(gray[x, y]) - seed_val) <= threshold:
```

```
            region[x, y] = 255
            for dx, dy in neighbors:
                nx, ny = x + dx, y + dy
                if 0 <= nx < h and 0 <= ny < w and not visited[nx, ny]:
                    q.append((nx, ny))

    return region
```

Listing 12: Region Growing so với cường độ tại seed ban đầu

6.5. Tự cài đặt Region Growing: so với trung bình vùng hiện tại

Biến thể này linh hoạt hơn vì trung bình vùng thay đổi khi vùng lớn dần.

```
def region_growing_region_mean(gray, seed, threshold=8, connectivity=4)
    :
    h, w = gray.shape
    visited = np.zeros((h, w), dtype=bool)
    region = np.zeros((h, w), dtype=np.uint8)

    q = deque([seed])
    region_sum = 0.0
    region_count = 0

    if connectivity == 8:
        neighbors = [(-1, -1), (-1, 0), (-1, 1),
                     (0, -1),           (0, 1),
                     (1, -1),  (1, 0),  (1, 1)]
    else:
        neighbors = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    while q:
        x, y = q.popleft()
        if visited[x, y]:
            continue
        visited[x, y] = True

        current_mean = gray[seed] if region_count == 0 else (region_sum
            / region_count)
        if abs(float(gray[x, y]) - float(current_mean)) <= threshold:
            region[x, y] = 255
            region_sum += float(gray[x, y])
            region_count += 1
            for dx, dy in neighbors:
                nx, ny = x + dx, y + dy
                if 0 <= nx < h and 0 <= ny < w and not visited[nx, ny]:
                    q.append((nx, ny))

    return region
```

Listing 13: Region Growing so với trung bình vùng hiện tại

6.6. Seed tốt và seed xấu

Đây là một điểm phải được làm rõ bằng hình ảnh và thí nghiệm. Nếu seed nằm trong phần điển hình của đối tượng, cường độ sáng tại đó đại diện tốt cho cả vùng, thuật toán dễ lan đúng. Nếu seed nằm gần biên, ở vùng bóng mờ, hoặc ở nền, kết quả có thể lan lệch hoàn toàn.

Một cách minh họa hiệu quả trong giờ học là:

- chọn 2 đến 3 seed khác nhau trên cùng một ảnh,
- giữ nguyên threshold,
- hiển thị overlay của từng kết quả lên ảnh gốc.

```
gray = read_gray_image('sample_images/lesion.png')
seed_good = (80, 90)
seed_bad = (30, 30)

mask_good = region_growing_seed_ref(gray, seed_good, threshold=10)
mask_bad = region_growing_seed_ref(gray, seed_bad, threshold=10)

over_good = overlay_mask_on_gray(gray, mask_good)
over_bad = overlay_mask_on_gray(gray, mask_bad)

show_images([gray, over_good, over_bad],
            ['Ảnh ốgc', 'Seed ốtt', 'Seed ấxu'],
            cols=3, figsize=(15, 5))
```

Listing 14: Ví dụ minh họa seed tốt và seed xấu

6.7. Gợi ý loại ảnh nên dùng để minh họa

Region Growing phù hợp để minh họa trên:

- ảnh y tế xám với vùng tổn thương tương đối đồng nhất,
- ảnh vật liệu hoặc ảnh hiển vi có hạt sáng trên nền tối,
- ảnh mà đối tượng có độ sáng khá ổn định trong nội vùng.

Không nên chọn ngay từ đầu những ảnh có texture mạnh hoặc biên quá mờ, vì các bạn sẽ khó quan sát được bản chất của seed và tiêu chí vùng.

6.8. Bài tập mở rộng cho Region Growing

Bài R1: Ảnh hưởng của vị trí seed

Yêu cầu: Chọn ít nhất ba seed: một seed tốt ở trung tâm đối tượng, một seed gần biên, một seed ở nền. Chạy cùng một giá trị threshold.

Đầu vào: Ảnh xám có một đối tượng nổi bật trên nền.

Đầu ra: Ba mask và ba ảnh overlay.

Mục đích: Chứng minh bằng thực nghiệm rằng seed là yếu tố quyết định.

So sánh cần làm: So sánh vùng bị lan thừa, vùng bị lan thiếu, và khả năng bao phủ đối tượng.

Bài R2: So sánh hai tiêu chí đồng nhất

Yêu cầu: So sánh biến thể dùng intensity tại seed với biến thể dùng trung bình vùng.

Đầu vào: Ảnh trong đó đối tượng có biến đổi sáng nhẹ từ trong ra ngoài.

Đầu ra: Hai mask tương ứng.

Mục đích: Hiểu rằng tiêu chí so sánh ảnh hưởng mạnh đến sự phát triển vùng.

So sánh cần làm: Nhận xét biến thể nào bám vùng tốt hơn khi đối tượng không đồng nhất hoàn toàn.

Bài R3: Kết hợp edge để chặn lan vùng

Yêu cầu: Dùng Canny hoặc Sobel để tạo biên; trong quá trình growing, không cho phép lan qua các pixel có biên mạnh.

Đầu vào: Ảnh có biên tương đối rõ.

Đầu ra: Mask growing thường và mask growing có ràng buộc biên.

Mục đích: Liên kết kiến thức edge detection với region-based segmentation.

So sánh cần làm: Xem biên có giúp chặn lan sai hay làm vùng bị thiếu không.

Bài R4: Tự động chọn seed

Yêu cầu: Đề xuất một cách chọn seed tự động, ví dụ lấy điểm sáng nhất trong một ROI, hoặc lấy cực đại cục bộ sau khi lọc.

Đầu vào: Ảnh có một hoặc vài đối tượng sáng hơn nền.

Đầu ra: Seed tự động và mask kết quả.

Mục đích: Chuyển từ minh họa thủ công sang một pipeline bán tự động.

7. Phương pháp 3: Watershed và Distance Transform

7.1. Ý tưởng trực quan của watershed

Watershed xem ảnh như một địa hình: nơi tối hoặc thấp là thung lũng, nơi sáng hoặc cao là đỉnh. Nếu ta "đổ nước" từ các marker ban đầu, nước từ các vùng khác nhau sẽ dâng lên; đường phân cách giữa các lưu vực chính là biên segmentation.

Trong thực hành, watershed thường được dùng để tách các đối tượng chạm nhau hoặc dính nhau. Ví dụ kinh điển là tách đồng xu, hạt, tế bào, hồng cầu dính nhau.

7.2. Distance Transform là gì và tại sao quan trọng?

Với một ảnh nhị phân trong đó vùng đối tượng có giá trị 1 và nền là 0, **distance transform** gán cho mỗi pixel trong đối tượng một giá trị bằng khoảng cách tới biên gần nhất. Ký hiệu đơn giản:

$$D(p) = \min_{q \in \partial R} d(p, q),$$

trong đó ∂R là biên của vùng đối tượng.

Ý nghĩa thực hành rất quan trọng:

- pixel càng ở sâu trong đối tượng, giá trị distance càng lớn,
- đỉnh của distance map thường nằm gần tâm hình học của đối tượng,
- nếu có hai vật thể dính nhau, distance map thường tạo ra hai đỉnh, từ đó ta có thể tạo marker riêng cho từng vật thể.

Nói cách khác, **distance transform biến bài toán "vật thể dính nhau" thành bài toán "tìm các đỉnh bên trong mỗi vật thể"**. Đây là lý do nó cực kỳ hữu ích trong watershed marker-based.

7.3. Pipeline đầy đủ cho Watershed dùng marker từ distance transform

Pipeline gợi ý cho Watershed

Ảnh xám → threshold hoặc Otsu → morphology để làm sạch mask → distance transform → threshold trên distance map để lấy sure foreground → tạo markers → watershed → đường biên và label của từng đối tượng.

7.4. Code đầy đủ cho pipeline Watershed với OpenCV

```
def watershed_from_binary(gray, blur_ksize=3, dist_ratio=0.45,
                          open_iter=2, dilate_iter=2):
    """ả
    Tr ềv các ếkt ảqu trung gian đểễ d ảging ạdy và ựtrc quan hóa.
    """
    if blur_ksize > 1:
        gray_blur = cv2.GaussianBlur(gray, (blur_ksize, blur_ksize), 0)
    else:
        gray_blur = gray.copy()

    # 1) Threshold Otsu
    _, binary = cv2.threshold(gray_blur, 0, 255,
                              cv2.THRESH_BINARY + cv2.THRESH_OTSU)

```

```
    # ếNu đối ượtng ốti trên ềnn sáng thì có ểth ầcn đảo ượngc:
    # binary = 255 - binary

    # 2) Opening để ạ loi ễnhiu ỏnh
    kernel = np.ones((3, 3), np.uint8)
    opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel,
                               iterations=open_iter)

    # 3) Sure background ằbng dilation
    sure_bg = cv2.dilate(opening, kernel, iterations=dilate_iter)

    # 4) Distance transform trên foreground
    dist_map = cv2.distanceTransform(opening, cv2.DIST_L2, 5)

    # 5) Sure foreground ằbng threshold trên distance map
    _, sure_fg = cv2.threshold(dist_map,
                               dist_ratio * dist_map.max(),
                               255, 0)
    sure_fg = sure_fg.astype(np.uint8)

    # 6) Vùng ưcha ếbit = ềnn ắchc ắchn - ềtin ảcnh ắchc ắchn
    unknown = cv2.subtract(sure_bg, sure_fg)

    # 7) Gán marker ừt connected components
    n_labels, markers = cv2.connectedComponents(sure_fg)
    markers = markers + 1  # ềnn không còn là 0
    markers[unknown == 255] = 0

    # 8) Watershed yêu ầcu ảnh 3 kênh
    color_input = cv2.cvtColor(gray, cv2.COLOR_GRAY2BGR)
    markers_ws = cv2.watershed(color_input, markers.copy())

    return {
        'gray_blur': gray_blur,
        'binary': binary,
        'opening': opening,
        'sure_bg': sure_bg,
        'dist_map': dist_map,
        'sure_fg': sure_fg,
        'unknown': unknown,
        'markers_before': markers,
        'markers_after': markers_ws
    }
```

Listing 15: Watershed marker-based từ ảnh xám

7.5. Hiển thị toàn bộ các kết quả trung gian

```
gray = read_gray_image('sample_images/coins_touching.png')
out = watershed_from_binary(gray, dist_ratio=0.42)

show_images(
```

```
    [out['gray_blur'], out['binary'], out['opening'], out['sure_bg'],
       out['sure_fg'], out['unknown']],
    ['Blur', 'Binary', 'Opening', 'Sure BG', 'Sure FG', 'Unknown'],
    cols=3, figsize=(14, 8), cmap='gray'
)

show_distance_map(out['dist_map'], title='Distance map')
show_markers(out['markers_before'], title='Markers before watershed')
show_markers(out['markers_after'], title='Markers after watershed')

overlay_ws = draw_watershed_boundaries(gray, out['markers_after'])
show_images([gray, overlay_ws], ['Ảnh ốgc', 'Biên watershed'], cols=2,
   figsize=(10, 5))
```

Listing 16: Hiển thị các bước trung gian của Watershed

7.6. Marker tốt và marker xấu

Trong watershed, marker có vai trò giống seed nhưng mạnh hơn: marker xác định các "nguồn ngập nước" ban đầu. Nếu marker quá nhỏ, quá ít, hoặc đặt sai vị trí, các đối tượng có thể bị nhập lại hoặc vỡ sai. Nếu marker được lấy tốt từ distance transform, mỗi đối tượng thường có một marker riêng gần tâm của nó.

Một thực nghiệm nên làm trong giờ học là thay đổi ngưỡng `dist_ratio`. Nếu ngưỡng quá thấp, sure foreground có thể dính nhau. Nếu ngưỡng quá cao, marker bị thu nhỏ quá mức hoặc mất hẳn. Kết quả watershed vì thế sẽ thay đổi rõ ràng.

7.7. Gợi ý loại ảnh nên dùng để minh họa

Watershed đặc biệt phù hợp với:

- ảnh đồng xu chạm nhau,
- ảnh hạt, viên thuốc, quả giống nhau đặt sát nhau,
- ảnh tế bào, RBC/WBC, đối tượng y sinh dính nhau,
- ảnh trong đó threshold nhị phân tương đối tốt nhưng các thành phần liên thông bị dính.

7.8. Bài tập mở rộng cho Watershed

Bài W1: Ảnh hưởng của ngưỡng trên distance map

Yêu cầu: Thử ít nhất ba giá trị `dist_ratio`, ví dụ 0.30, 0.45, 0.60.

Đầu vào: Ảnh có nhiều vật thể dính nhau.

Đầu ra: Distance map, sure foreground, marker và kết quả watershed cho từng trường hợp.

Mục đích: Hiểu rõ distance transform không chỉ để xem, mà được dùng trực tiếp để tạo marker.

So sánh cần làm: Đếm xem số đối tượng tách ra được là bao nhiêu; nhận xét trường hợp nào over-segmentation hoặc under-segmentation.

Bài W2: Ảnh hưởng của morphology trước watershed

Yêu cầu: So sánh các tổ hợp opening/closing khác nhau trước khi tính distance transform.

Đầu vào: Ảnh có nhiễu hạt hoặc mask threshold ban đầu còn lỗ nhỏ.

Đầu ra: Binary mask sau morphology, distance map và kết quả cuối.

Mục đích: Liên kết trực tiếp kiến thức morphology với watershed.

So sánh cần làm: Nhận xét lỗ nhỏ, điểm nhiễu, và vùng dính thay đổi như thế nào sau mỗi phép toán hình thái.

Bài W3: Marker thủ công và marker tự động

Yêu cầu: Với cùng một ảnh, hãy tạo hai hệ marker: một hệ marker thủ công (đặt vài điểm trong từng vật thể), và một hệ marker tự động từ distance transform.

Đầu vào: Ảnh có số lượng đối tượng vừa phải để có thể đặt marker bằng tay.

Đầu ra: Hai kết quả watershed tương ứng.

Mục đích: Thấy ưu nhược điểm giữa cách làm tương tác và tự động.

So sánh cần làm: Đánh giá độ ổn định, công sức thao tác, và chất lượng tách đối tượng.

Bài W4: Kết hợp gradient làm bề mặt watershed

Yêu cầu: Thay vì dùng trực tiếp ảnh xám, hãy thử tạo gradient magnitude từ Sobel/Canny rồi dùng gradient làm bề mặt để watershed.

Đầu vào: Ảnh có biên tương đối rõ.

Đầu ra: Kết quả watershed theo hai kiểu đầu vào khác nhau.

Mục đích: Hiểu một cách sâu hơn về “địa hình” trong watershed.

8. Phương pháp 4: Split–Merge segmentation

8.1. Ý tưởng

Split–Merge dựa trên quan điểm rằng một vùng tốt là vùng ”đồng nhất”. Nếu một vùng chưa đồng nhất thì chia nhỏ ra (split). Sau đó, những vùng con kề nhau mà đủ giống nhau có thể được gộp lại (merge).

Đây là một phương pháp rất tốt để các bạn hiểu khái niệm phân đoạn theo vùng một cách có hệ thống, vì nó gắn chặt với ý tưởng kiểm tra tính đồng nhất trên từng miền ảnh.

8.2. Tiêu chí đồng nhất

Trong bài lab này, ta dùng tiêu chí đơn giản là phương sai mức xám của một vùng phải nhỏ hơn ngưỡng đã cho:

$$\mathrm{Var}(R) < T.$$

Nếu phương sai lớn, nghĩa là vùng còn quá không đồng nhất, ta chia vùng đó thành bốn phần bằng nhau theo kiểu quadtree.

8.3. Pipeline gợi ý cho Split–Merge

Pipeline gợi ý cho Split–Merge

Ảnh xám → kiểm tra độ đồng nhất trên toàn ảnh → nếu chưa đồng nhất thì split thành 4 ô → đệ quy trên từng ô → sau khi split xong thì xét merge các ô kề nhau nếu mức xám trung bình gần nhau.

8.4. Code đầy đủ phần split theo quadtree

Để giữ bài lab rõ ràng, ta tách phần **split** ra trước. Đây không phải pseudo-code, mà là mã chạy được.

```python
def quadtree_split(gray, min_size=16, var_thresh=80.0):
    """ả
    Tr ềv danh sách các hình ữch ậnht (x, y, w, h)ể
    biu ễdin các ô lá ủca quadtree.
    """
    h, w = gray.shape
    rectangles = []

    def recurse(x, y, hh, ww):
        region = gray[x:x+hh, y:y+ww]
        variance = float(np.var(region))

        if hh <= min_size or ww <= min_size or variance < var_thresh:
            rectangles.append((x, y, ww, hh))
            return

        h2 = hh // 2
        w2 = ww // 2

        if h2 == 0 or w2 == 0:
            rectangles.append((x, y, ww, hh))
            return

        recurse(x, y, h2, w2)
        recurse(x, y + w2, h2, ww - w2)
        recurse(x + h2, y, hh - h2, w2)
        recurse(x + h2, y + w2, hh - h2, ww - w2)

```

```
    recurse(0, 0, h, w)
    return rectangles
```

Listing 17: Tách ảnh theo quadtree dựa trên ngưỡng phương sai

8.5. Gán nhãn các ô split để trực quan

```
def quadtree_rectangles_to_label_map(shape, rectangles):
    h, w = shape
    label_map = np.zeros((h, w), dtype=np.int32)
    for idx, (x, y, ww, hh) in enumerate(rectangles, start=1):
        label_map[x:x+hh, y:y+ww] = idx
    return label_map
```

Listing 18: Tạo label map từ các ô quadtree

8.6. Merge các ô kề nhau theo trung bình mức xám

Việc merge đầy đủ có thể khá dài; dưới đây là một phiên bản đơn giản, đủ để giảng dạy, trong đó ta gộp các ô kề nhau nếu trung bình mức xám chênh nhau không quá một ngưỡng cho trước.

```
def merge_adjacent_rectangles(gray, rectangles, mean_thresh=8.0):
    """
    Gom các ô thành nhóm ằbng BFS trên đồị th ềk nhau.ế
    Kt ảqu là danh sách nhóm các ỉch ốs ô.
    """
    means = []
    for (x, y, ww, hh) in rectangles:
        means.append(float(np.mean(gray[x:x+hh, y:y+ww])))

    def are_adjacent(r1, r2):
        x1, y1, w1, h1 = r1
        x2, y2, w2, h2 = r2
        # ểKim tra ềk ạcnh theo ụtrc ngang ặhoc ọdc
        horiz_touch = (x1 < x2 + h2 and x2 < x1 + h1) and (y1 + w1 ==
            y2 or y2 + w2 == y1)
        vert_touch = (y1 < y2 + w2 and y2 < y1 + w1) and (x1 + h1 == x2
            or x2 + h2 == x1)
        return horiz_touch or vert_touch

    n = len(rectangles)
    visited = [False] * n
    groups = []

    for i in range(n):
        if visited[i]:
            continue
        q = deque([i])
        visited[i] = True
        group = [i]
```

```python

        while q:
            u = q.popleft()
            for v in range(n):
                if not visited[v] and are_adjacent(rectangles[u],
                    rectangles[v]):
                    if abs(means[u] - means[v]) <= mean_thresh:
                        visited[v] = True
                        q.append(v)
                        group.append(v)
        groups.append(group)

    return groups


def groups_to_label_map(shape, rectangles, groups):
    h, w = shape
    label_map = np.zeros((h, w), dtype=np.int32)
    for label, group in enumerate(groups, start=1):
        for idx in group:
            x, y, ww, hh = rectangles[idx]
            label_map[x:x+hh, y:y+ww] = label
    return label_map
```

Listing 19: Merge đơn giản các ô kề nhau theo trung bình mức xám

8.7. Ví dụ minh họa Split–Merge

```python
gray = read_gray_image('sample_images/texture_blocks.png')
rects = quadtree_split(gray, min_size=16, var_thresh=120.0)
label_split = quadtree_rectangles_to_label_map(gray.shape, rects)

groups = merge_adjacent_rectangles(gray, rects, mean_thresh=10.0)
label_merge = groups_to_label_map(gray.shape, rects, groups)

draw_quadtree_boxes(gray, rects, title='Quadtree split')
show_images([colorize_label_map(label_split), colorize_label_map(
    label_merge)],
            ['Split labels', 'After merge'], cols=2, figsize=(12, 5),
                cmap='gray')
```

Listing 20: Ví dụ chạy Split–Merge và hiển thị kết quả

8.8. Gợi ý loại ảnh nên dùng để minh họa

Split–Merge nên minh họa trên các ảnh:

- có các khối vùng tương đối lớn và khá đồng nhất,
- có nền chia mảng rõ, ví dụ ảnh tài liệu, ảnh vật liệu có các mảng cường độ khác nhau,

- ảnh texture yếu hoặc texture theo vùng lớn.

Nếu dùng ngay ảnh có nhiều chi tiết nhỏ và biên cong phức tạp, các bạn sẽ thấy quá nhiều ô split và khó hiểu bản chất.

8.9. Bài tập mở rộng cho Split–Merge

Bài S1: Ảnh hưởng của ngưỡng phương sai

Yêu cầu: Thử ít nhất ba giá trị `var_thresh` khác nhau.

Đầu vào: Ảnh có nhiều vùng đồng nhất ở các mức sáng khác nhau.

Đầu ra: Quadtree rectangles và label map tương ứng.

Mục đích: Hiểu rằng ngưỡng đồng nhất quyết định mức độ chia nhỏ của ảnh.

So sánh cần làm: Đếm số ô lá sinh ra và nhận xét vùng nào bị split quá mức.

Bài S2: Ảnh hưởng của kích thước tối thiểu

Yêu cầu: Thử `min_size = 8, 16, 32`.

Đầu vào: Ảnh có cả vùng lớn lẫn chi tiết nhỏ.

Đầu ra: Kết quả split và merge tương ứng.

Mục đích: Hiểu vai trò của điều kiện dừng trong đệ quy quadtree.

So sánh cần làm: Nhận xét mức độ chi tiết và độ mượt của phân đoạn.

Bài S3: So sánh có merge và không merge

Yêu cầu: Chạy split rồi hiển thị label map; sau đó chạy thêm merge và hiển thị label map mới.

Đầu vào: Ảnh có các vùng lớn nhưng hơi dao động sáng trong nội vùng.

Đầu ra: Hai label map.

Mục đích: Thấy rõ split chỉ là bước đầu, còn merge giúp phục hồi các vùng lớn có ý nghĩa hơn.

Bài S4: Kết hợp tiền xử lý trước Split–Merge

Yêu cầu: So sánh kết quả split trên ảnh gốc và ảnh đã lọc Gaussian.

Đầu vào: Ảnh có nhiễu nhẹ.

Đầu ra: Hai sơ đồ quadtree.

Mục đích: Thấy nhiễu làm tăng phương sai cục bộ và gây split không cần thiết.

9. So sánh tổng hợp giữa các phương pháp

Phương pháp	Đầu vào chính	Ưu điểm	Hạn chế
K-means	Pixel dạng vector đặc trưng, ví dụ $[I]$ hoặc $[I, x, y]$	Đơn giản, dễ cài đặt, trực quan, chạy nhanh	Không dựa trực tiếp vào tính liên thông; nhạy với số cụm K; có thể gộp vùng ở xa nhau nếu cùng mức sáng
Region Growing	Ảnh xám + seed + ngưỡng đồng nhất	Trực quan, có tính liên thông vùng, dễ giải thích trong giảng dạy	Phụ thuộc mạnh vào seed; dễ lan sai khi có nhiễu hoặc biên mờ
Watershed	Ảnh hoặc mask nhị phân + marker	Rất mạnh khi tách đối tượng dính nhau; kết nối chặt với distance transform và morphology	Rất nhạy với marker; dễ over-segmentation nếu marker hoặc bề mặt không tốt
Split–Merge	Ảnh xám + tiêu chí đồng nhất theo vùng	Giúp hiểu rõ khái niệm vùng đồng nhất; có cấu trúc phân cấp	Cài đặt phức tạp hơn; biên vùng thường khối chữ nhật nếu chỉ dùng quadtree cơ bản

10. Các bài tập tổng hợp cho toàn bộ bài học

Phần này gồm các bài tập tổng hợp yêu cầu sinh viên phải phối hợp nhiều kỹ thuật đã học trong suốt học phần, thay vì chỉ chạy riêng một thuật toán.

Bài T1: Thiết kế pipeline phân đoạn hoàn chỉnh cho ảnh đồng xu chạm nhau

Yêu cầu: Xây dựng pipeline đầy đủ để tách từng đồng xu riêng biệt trên ảnh có các đồng xu chạm nhau.

Đầu vào: Một ảnh xám hoặc ảnh màu chuyển sang xám được, trong đó nhiều đồng xu sáng trên nền tối hoặc ngược lại.

Đầu ra:

- mask nhị phân ban đầu,
- distance map,
- markers,
- label map cuối cùng hoặc ảnh biên watershed chồng lên ảnh gốc.

Mục đích: Luyện khả năng nối các bước đã học: threshold, morphology, distance transform, watershed.

Ảnh mẫu nên chọn: Ảnh đồng xu hoặc hạt tròn có kích thước tương tự nhau.

So sánh cần làm: So sánh với cách chỉ threshold rồi connected components để thấy vì sao watershed cần thiết khi các vật thể dính nhau.

Bài T2: So sánh K-means, Region Growing và Watershed trên cùng một bài toán

Yêu cầu: Chọn một ảnh xám có đối tượng nổi bật, áp dụng ba phương pháp: K-means, Region Growing, Watershed.

Đầu vào: Một ảnh có ít nhất một đối tượng chính và nền tương đối phân biệt.

Đầu ra: Kết quả phân đoạn của cả ba phương pháp, kèm nhận xét bằng văn bản.

Mục đích: Không chỉ biết chạy thuật toán, mà còn biết chọn thuật toán phù hợp cho một bài toán cụ thể.

Ảnh mẫu nên chọn: Ảnh lesion đơn giản, ảnh hạt, ảnh vật thể công nghiệp trên nền sạch.

So sánh cần làm:

- mức độ bám đúng đối tượng,
- độ nhạy với tham số,
- độ dễ triển khai,
- khi nào thất bại và vì sao.

Bài T3: Thí nghiệm Seed/Marker tốt và không tốt

Yêu cầu: Với cùng một ảnh, tạo hai hoặc ba seed khác nhau cho Region Growing; đồng thời tạo hai bộ marker khác nhau cho Watershed (ví dụ thay đổi ngưỡng distance transform).

Đầu vào: Ảnh có đối tượng rõ nhưng biên không hoàn toàn đồng đều.

Đầu ra: Bộ kết quả thể hiện rõ trường hợp tốt và xấu.

Mục đích: Làm nổi bật một trong những thông điệp quan trọng nhất của bài học: **seed/marker không phải chi tiết phụ, mà là yếu tố quyết định kết quả.**

Ảnh mẫu nên chọn: Ảnh y sinh, ảnh hạt hoặc ảnh vật thể có vùng sáng tương đối đồng nhất.

So sánh cần làm: Nhận xét bằng hình và mô tả ngắn về lan thiếu, lan thừa, nhập sai vùng, tách sai số lượng đối tượng.

Bài T4: Kết hợp preprocessing với segmentation

Yêu cầu: Lấy một ảnh nhiễu nhẹ; áp dụng segmentation theo cùng một thuật toán trước và sau khi lọc Gaussian hoặc median.

Đầu vào: Ảnh chứa nhiễu hoặc nền không đều.

Đầu ra: Hai hoặc nhiều kết quả segmentation tương ứng.

Mục đích: Nhấn mạnh rằng segmentation không đứng riêng lẻ; tiền xử lý có thể thay đổi chất lượng phân đoạn rất mạnh.

Ảnh mẫu nên chọn: Ảnh hiển vi, ảnh vật liệu, ảnh công nghiệp có nhiễu lốm đốm.

So sánh cần làm: Quan sát số vùng nhiễu giả, mức độ rời rạc của biên, số vùng split không cần thiết.

Bài T5: Thiết kế hệ thống nhỏ cho một ứng dụng thực tế

Yêu cầu: Chọn một bài toán trong ba hướng sau và thiết kế pipeline hoàn chỉnh:

- tách tế bào hoặc hạt trong ảnh hiển vi,
- tách sản phẩm/linh kiện trên nền băng tải,
- tách vùng bất thường sáng/tối trên ảnh xám y tế đơn giản.

Đầu vào: Tự chọn bộ ảnh nhỏ gồm 3–5 ảnh cùng loại.

Đầu ra:

- mô tả pipeline,
- code,
- ảnh kết quả,
- phần nhận xét phương pháp đã chọn và lý do chọn.

Mục đích: Rèn năng lực chọn thuật toán, thiết kế pipeline và giải thích quyết định kỹ thuật.

So sánh cần làm: Nên thử ít nhất hai cấu hình hoặc hai phương pháp và giải thích vì sao cấu hình cuối là phù hợp hơn.

11. Hướng dẫn viết báo cáo sau buổi lab

Trong báo cáo, sinh viên không nên chỉ chụp kết quả cuối cùng. Một báo cáo tốt cần trình bày:

1. ảnh đầu vào và đặc điểm của ảnh,
2. pipeline đã chọn,
3. các kết quả trung gian quan trọng như mask, distance map, markers, cluster map, quadtree,
4. phân tích tham số,
5. so sánh giữa các phương pháp,
6. kết luận: phương pháp nào phù hợp cho kiểu ảnh nào và vì sao.

12. Kết luận của bài học

Sau khi hoàn thành bài lab này, sinh viên cần nắm được các ý chính sau:

- Phân đoạn dựa trên vùng không chỉ là "chạy thuật toán", mà là một quá trình thiết kế pipeline.
- K-means, Region Growing, Watershed và Split–Merge tuy đều thuộc nhóm region-based nhưng dựa trên những giả thiết rất khác nhau.
- Chọn đúng đặc trưng đầu vào, đúng seed, đúng marker và đúng bước tiền xử lý quan trọng không kém bản thân thuật toán.
- Distance transform có ý nghĩa thực hành rất lớn trong bài toán tách các đối tượng chạm nhau.
- Việc hiển thị các kết quả trung gian là bắt buộc nếu muốn hiểu và phân tích thuật toán một cách sâu sắc.